

JAVA . CLOUD . ORACLE . SPRINGBOOT

Oracle Cloud Infrastructure: What Java Teams Overlook

OKE, Autonomous Database, and Vault for Spring Boot
workloads

Said Olano

Head of Engineering . FinTech & Digital Banking

Oracle Cloud Infrastructure: Cosa i team Java si perdono

La maggior parte dei team Java sceglie per default AWS o Azure senza mai valutare il costo di Oracle Cloud Infrastructure (OCI). È comprensibile: la mentalità diffusa non è quella. Ma se lavori con carichi Java, specialmente qualcosa che tocca Oracle Database, OCI ha alcuni vantaggi facili da trascurare finché non ti ritrovi a pagare costi di uscita dati o ad avviare l'ennesimo cluster Kubernetes gestito altrove.

Il problema di dare per scontato che "tutti usano AWS"

Le decisioni sul cloud nella maggior parte delle organizzazioni di ingegneria vengono prese una sola volta, presto, e raramente vengono riviste. Va bene finché non arriva la fattura. Due cose colgono di sorpresa i team sugli altri provider: i costi di trasferimento dati in uscita e il prezzo di mantenere calcolo sempre attivo per carichi che non ne hanno bisogno. Nessuno dei due è esclusivo di un singolo fornitore, ma il modello di prezzo di OCI — piatto, prevedibile e notevolmente più economico sull'uscita dati — merita una seconda occhiata se non lo controlli dall'ultimo confronto tra provider.

Cosa è davvero utile per uno stack Java

Alcuni servizi OCI si sovrappongono direttamente a ciò di cui un tipico team Spring Boot / microservizi ha già bisogno:

- **Autonomous Database** — un database Oracle che si auto-aggiorna e si auto-ottimizza. Se la tua app comunica già con Oracle via JDBC, questo elimina gran parte del carico di lavoro da DBA.
- **Container Engine for Kubernetes (OKE)** — un'offerta Kubernetes gestita che è una destinazione diretta per gli stessi Helm chart e manifest che già usi altrove.

- **OCI Functions** — serverless, costruito sul progetto open-source Fn Project, il che significa che è portabile invece di essere un vicolo cieco proprietario.
- **Vault** — gestione centralizzata di segreti e chiavi, nella stessa forma di AWS Secrets Manager o HashiCorp Vault, se preferisci non gestirne uno tuo.

Niente di tutto questo richiede di riscrivere nulla. Un servizio Spring Boot che già esternalizza la configurazione del datasource può puntare a un'istanza di Autonomous Database con una modifica di una riga:

```
spring:
  datasource:
    url: jdbc:oracle:thin:@myatp_high?TNS_ADMIN=/opt/oracle/wallet
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    driver-class-name: oracle.jdbc.OracleDriver
```

La connessione basata su wallet gestisce TLS e failover per te, il che è una cosa in meno da costruire a mano in un HikariConfig.

Dove non convince

L'ecosistema e le dimensioni della community di OCI non sono vicini a quelli di AWS. Meno moduli Terraform, meno risposte su Stack Overflow, meno integrazioni di terze parti pronte all'uso. Anche la presenza multi-regione è più limitata, quindi se i tuoi requisiti di latenza sono globali e specifici, controlla la copertura delle regioni prima di impegnarti. E se il tuo team ha anni di strumenti e abitudini specifiche per AWS, il costo di una migrazione completa raramente si giustifica solo per il risparmio sui costi di uscita dati.

Dove vale la pena valutarlo

Il caso a favore di OCI è più forte quando sei già un'azienda che usa Oracle Database, quando i tuoi carichi di lavoro sono a lettura intensiva con molto traffico in uscita, o quando stai costruendo qualcosa da zero e la prevedibilità dei costi conta più dell'ampiezza dell'ecosistema. Raramente è un sostituto completo di AWS o Azure — più spesso si guadagna un posto in una strategia multi-cloud per i carichi specifici in cui è davvero valido.

Avete già valutato OCI per un carico di lavoro Java, oppure AWS/Azure resta la scelta scontata nel vostro team?